

Introducción a Python para Ingeniería, Datos y Economía

A. Soleal

2026-09-22

Tabla de contenidos

Introducción a Python para Ingeniería, Datos y Economía	3
Prólogo	3
Cómo usar este libro	3
1 Introducción a Python y Google Colab	4
1.1 ¿Qué es Python y por qué lo usaremos en Google Colab?	4
1.2 Tu primer programa	4
1.3 Python como calculadora	5
1.4 Los errores son tus amigos	6
1.5 Guardar tu trabajo	6
1.6 Ejercicios	6
1.7 Resumen	7
2 Fundamentos: variables y tipos de datos	8
2.1 ¿Qué es una variable?	8
2.2 Reglas para nombrar variables	8
2.3 Los cuatro tipos de datos básicos	9
2.4 Convertir entre tipos	10
2.5 Pedir datos al usuario: <code>input()</code>	10
2.6 Imprimir con formato: las f-strings	11
2.7 Comentarios: notas para humanos	11
2.8 Ejercicios	12
2.9 Resumen	12
3 Capítulo 3: Control de flujo	13
4 Capítulo 4: Estructuras de datos	14
5 Capítulo 5: NumPy y gráficas	15
6 Capítulo 6: pandas	16
7 Capítulo 7: Economía y administración	17
8 Capítulo 8: Introducción a big data	18

Introducción a Python para Ingeniería, Datos y Economía

John Leal

Prólogo

Este libro está pensado para estudiantes y profesionales de ingeniería, análisis de datos y economía que desean aprender a programar en Python desde cero. No se asume ninguna experiencia previa en programación: los conceptos se introducen de forma progresiva, con ejemplos cercanos a la ingeniería, la estadística y la economía, para que cada tema tenga un contexto claro y aplicable.

Todo el código está diseñado para ejecutarse en Google Colab, el entorno gratuito de notebooks de Google. No necesitas instalar nada en tu computadora: basta un navegador y una cuenta de Google. Cada capítulo contiene bloques de código listos para copiar y pegar en una celda de Colab, de modo que puedes leer, ejecutar y experimentar en el mismo momento.

Al terminar este libro habrás aprendido los fundamentos del lenguaje Python —variables, tipos de datos, operadores y control de flujo—, las estructuras de datos esenciales, el cálculo numérico con NumPy, la visualización de gráficas, el análisis de datos con pandas, y una introducción a las aplicaciones en economía, administración y big data.

Cómo usar este libro

Cada bloque de código de este libro se copia y se pega tal cual en una celda de Google Colab. Para ello, abre colab.research.google.com, crea un notebook nuevo y copia los bloques de cada capítulo en orden, ejecutándolos con el botón de reproducción (o con **Ctrl + Enter**).

Te recomendamos crear un notebook por capítulo, ejecutar cada bloque paso a paso y modificar los valores de los ejemplos para comprobar qué ocurre: aprender a programar se logra escribiendo y ejecutando código, no solo leyéndolo.

1 Introducción a Python y Google Colab

1.1. ¿Qué es Python y por qué lo usaremos en Google Colab?

Python es un lenguaje de programación de propósito general, creado con una idea central: que el código se lea casi como texto en inglés. Esa claridad lo convirtió en el lenguaje más usado del mundo en ciencia de datos, ingeniería, economía e inteligencia artificial.

¿Por qué Python y no otro lenguaje? Porque con las mismas diez líneas de código que en otros lenguajes apenas configurarían el entorno, en Python ya estás analizando datos reales, graficando una función o calculando el valor presente neto de un proyecto.

En este libro trabajaremos con **Google Colab**: un entorno gratuito de Google que corre en el navegador. No necesitas instalar nada en tu computador; solo una cuenta de Google. Cada bloque de código de este libro está diseñado para que lo copies, lo pegues en una celda de Colab y lo ejecutes.

1.2. Tu primer programa

1. Abre colab.research.google.com en tu navegador.
2. Haz clic en **Nuevo notebook**.
3. En la primera celda, escribe (o copia) lo siguiente:

```
print("Hola, mundo")
```

4. Ejecuta la celda con el botón  o con las teclas **Shift + Enter**.

En pantalla aparecerá:

```
Hola, mundo
```

¿Qué acaba de pasar? `print()` es una **función**: una instrucción que le dice a Python “muestra esto en pantalla”. El texto entre comillas se llama **cadena de caracteres** (*string*). Acabas de escribir y ejecutar tu primer programa.

Nota

Un notebook de Colab tiene dos tipos de celdas: **código** (se ejecuta) y **texto** (se lee, como este párrafo). Agrégalas con los botones + Código y + Texto.

1.3. Python como calculadora

Antes de programar “en serio”, usa Python como una calculadora. Copia y ejecuta cada línea:

```
2 + 3
```

```
10 / 4
```

```
2 ** 10
```

```
17 % 5
```

Los operadores básicos son:

Tabla 1.1: Operadores aritméticos básicos de Python

Operador	Significado	Ejemplo	Resultado
+	Suma	2 + 3	5
-	Resta	7 - 4	3
*	Multiplicación	3 * 5	15
/	División	10 / 4	2.5
**	Potencia	2 ** 10	1024
//	División entera	17 // 5	3
%	Residuo (módulo)	17 % 5	2

Un ejemplo con sentido económico: si inviertes \$1.000.000 a una tasa del 12% anual durante 5 años, el valor futuro es:

```
1000000 * (1 + 0.12) ** 5
```

Observa que Python respeta el orden de las operaciones (primero paréntesis, luego potencias, luego multiplicación y división, al final suma y resta), igual que en álgebra.

1.4. Los errores son tus amigos

Escribe esto en una celda y ejecútalo:

```
print("Hola"
```

Verás algo como:

```
SyntaxError: incomplete input
```

No te alarmes: un error no rompe nada. Python te está diciendo **qué** falló (**SyntaxError**: error de sintaxis) y en este caso es evidente: falta cerrar el paréntesis. Corrige y ejecuta de nuevo.



Tip

Regla de oro para todo el libro: **lee los errores de abajo hacia arriba**. La última línea del mensaje casi siempre dice exactamente qué salió mal.

1.5. Guardar tu trabajo

Colab guarda tus notebooks automáticamente en tu Google Drive (carpeta Colab Notebooks). Buenas prácticas desde el primer día:

- Renombra el notebook: clic en el título de arriba a la izquierda. Usa nombres sin espacios ni tildes: `capitulo01_introduccion.ipynb`.
- Para descargarlo: Archivo → Descargar → Descargar `.ipynb`.
- Crea un notebook por capítulo de este libro; te servirán como tu propio cuaderno de apuntes ejecutable.

1.6. Ejercicios

1. Ejecuta `print("Tu nombre")` con tu nombre real.
2. Calcula cuántos segundos tiene un año (365 días) usando solo operadores.
3. Calcula el valor futuro de \$5.000.000 al 8% anual durante 10 años.
4. Provoca a propósito un error quitando una comilla en un `print`, lee el mensaje y corrígelo.
5. **Mini-reto:** una hectárea produce 4,2 toneladas de café. ¿Cuántas toneladas producen 37 hectáreas? ¿Y si el precio es \$9.500.000 por tonelada, cuánto vale la cosecha total? Resuélvelo en una sola celda con dos operaciones.

1.7. Resumen

- Python es un lenguaje claro y es el estándar en datos e ingeniería.
- Colab te permite programar sin instalar nada, desde el navegador.
- `print()` muestra resultados en pantalla.
- Python funciona como calculadora con los operadores `+` `-` `*` `/` `**` `//` `%`.
- Los errores son mensajes de ayuda: léelos de abajo hacia arriba.

2 Fundamentos: variables y tipos de datos

2.1. ¿Qué es una variable?

Imagina una caja con una etiqueta. Dentro de la caja guardas algo (un número, un texto), y la etiqueta te permite encontrarla después. Eso es una **variable**: un nombre que guarda un valor.

En Python, guardar un valor en una variable se llama **asignar**, y se hace con el signo **=**:

```
precio = 9500
```

Lee esa línea así: “la variable `precio` guarda el valor 9500”. Ojo: el `=` de Python **no es** el igual de las matemáticas; no afirma que dos cosas sean iguales, sino que **guarda** el valor de la derecha en el nombre de la izquierda.

A partir de ahí, puedes usar la variable en cualquier cálculo:

```
precio = 9500
cantidad = 37
total = precio * cantidad
print(total)
```

La gran ventaja: si el precio cambia, solo modificas una línea y todo lo demás se recalcula solo. Cambia `precio` a 10200 y vuelve a ejecutar.

2.2. Reglas para nombrar variables

Python es estricto con los nombres. Estos son válidos:

```
precio_tonelada = 9500
temperatura2 = 25.4
nombre_finca = "La Esperanza"
```

Estos **no** son válidos (ejecútalos y lee el error):

```
2temperatura = 25          # no puede empezar con número
precio-tonelada = 9500     # no puede tener guiones
precio tonelada = 9500     # no puede tener espacios
```

i Nota

Convención de este libro: nombres en minúsculas, sin tildes, con palabras separadas por guion bajo: `valor_futuro`, `tasa_anual`, `numero_hectareas`. No es obligatorio, pero es el estilo que usa todo el mundo de Python.

2.3. Los cuatro tipos de datos básicos

Todo valor en Python tiene un **tipo**. Para empezar solo necesitas conocer cuatro:

Tipo	Nombre	Qué guarda	Ejemplo
<code>int</code>	Entero	Números sin decimales	37, -5, 1000000
<code>float</code>	Decimal	Números con decimales	4.2, 0.12, -3.5
<code>str</code>	Texto	Cadenas de caracteres	"café", "La Esperanza"
<code>bool</code>	Booleano	Verdadero o falso	True, False

Python detecta el tipo automáticamente según lo que guardes. Puedes preguntárselo con la función `type()`:

```
precio = 9500
print(type(precio))
```

```
rendimiento = 4.2
print(type(rendimiento))
```

```
nombre = "Ana"
print(type(nombre))
```

```
es_rentable = True
print(type(es_rentable))
```

Advertencia

Cuidado con el punto y la coma: en Python los decimales se escriben con **punto**, no con coma. Escribe 4.2, no 4,2. Si escribes 4,2 Python no dará un error obvio: pensará que son dos números separados. Es uno de los errores más comunes de quienes empiezan en países hispanos.

2.4. Convertir entre tipos

A veces necesitas transformar un tipo en otro. Se hace con funciones que tienen el nombre del tipo: `int()`, `float()`, `str()`.

El caso clásico: los números que vienen como texto no se pueden sumar:

```
a = "2"
b = "3"
print(a + b)
```

¿Sorprendido? El resultado es "23", no 5. Como `a` y `b` son **texto**, el `+` los une en lugar de sumarlos (pegar textos se llama **concatenar**). La solución es convertirlos primero:

```
a = "2"
b = "3"
print(int(a) + int(b))
```

2.5. Pedir datos al usuario: `input()`

Hasta ahora los valores estaban escritos en el código. Pero normalmente querrás preguntarle a la persona que usa tu programa. Eso se hace con `input()`:

```
nombre = input("¿Cómo te llamas? ")
print("Hola,", nombre)
```

Al ejecutar, el programa se detiene, muestra la pregunta y espera a que escribas algo y presiones **Enter**.

! Importante

Lo que `input()` devuelve siempre es texto (`str`), aunque la persona escriba un número. Si vas a hacer cuentas con esa respuesta, conviértela:

```
edad = input("¿Cuántos años tienes? ")
edad = int(edad)
print("El próximo año tendrás", edad + 1)
```

O en una sola línea, que es como lo verás en la práctica:

```
edad = int(input("¿Cuántos años tienes? "))
```

2.6. Imprimir con formato: las f-strings

Ya sabes que `print()` muestra cosas. Cuando quieras mezclar texto con variables de forma elegante, usa una **f-string**: una cadena con una `f` antes de las comillas, donde las variables van entre llaves `{}`:

```
precio = 9500
cantidad = 37
total = precio * cantidad
print(f"El total de la cosecha es ${total}")
```

Todo lo que está dentro de las llaves se evalúa. Puedes incluso hacer cuentas ahí dentro:

```
tasa = 0.12
capital = 1000000
print(f"En 5 años tendrás ${capital * (1 + tasa) ** 5}")
```

2.7. Comentarios: notas para humanos

Todo lo que escribas después de un `#` en una línea, Python lo ignora por completo. Sirve para dejar notas que expliquen tu código:

```
# Cálculo del valor futuro a 5 años
tasa = 0.12          # tasa anual
capital = 1000000    # inversión inicial en pesos
```

```
vf = capital * (1 + tasa) ** 5
print(f"Valor futuro: ${vf}")
```

Comenta tu código pensando en la persona que lo leerá en seis meses. Esa persona, casi siempre, serás tú.

2.8. Ejercicios

1. Crea tres variables con tu nombre, tu edad y tu estatura (en metros, con decimal). Imprímelas con una f-string en una sola frase.
2. Pregunta al usuario cuántas hectáreas tiene su finca y cuántas toneladas produce por hectárea. Calcula e imprime la producción total.
3. Pregunta al usuario su salario mensual y calcula cuánto gana al año. Imprime el resultado con una f-string.
4. Ejecuta esto, lee el error y corrígelo:

```
precio = input("Precio: ")
print(precio * 2)
```

(Pista: multiplica texto por dos y verás algo raro, pero no es lo que queremos).

5. **Mini-reto (economía):** pide al usuario un capital inicial, una tasa de interés anual (en porcentaje, ej. 12) y un número de años. Calcula el valor futuro con interés compuesto e imprímelo con formato: **Con \$X al Y% durante Z años, tendrás \$W.** Recuerda convertir la tasa de porcentaje a decimal dividiendo entre 100.

2.9. Resumen

- Una **variable** es un nombre que guarda un valor; se asigna con `=`.
- Los cuatro tipos básicos: `int` (enteros), `float` (decimales), `str` (texto), `bool` (verdadero/falso).
- Los decimales se escriben con **punto**: `4.2`.
- `input()` siempre devuelve texto; usa `int()` o `float()` para hacer cuentas.
- Las **f-strings** (`f"...{variable}..."`) mezclan texto y valores fácilmente.
- Los **comentarios** con `#` documentan tu código.

3 Capítulo 3: Control de flujo

En construcción.

4 Capítulo 4: Estructuras de datos

En construcción.

5 Capítulo 5: NumPy y gráficas

En construcción.

6 Capítulo 6: pandas

En construcción.

7 Capítulo 7: Economía y administración

En construcción.

8 Capítulo 8: Introducción a big data

En construcción.